```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!pip install kaggle shap lime
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (1.7.4.5)
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.48.0)
Collecting lime
  Downloading lime-0.2.0.1.tar.gz (275 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 275.7/275.7 kB 10.1 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.2.0)
Requirement already satisfied: certifi>=14.05.14 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2025.8.3)
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.4.3)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.10)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.5)
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: setuptools>=21.0.0 in /usr/local/lib/python3.12/dist-packages (from kaggle) (75.2.0)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.17.0)
Requirement already satisfied: text-unidecode in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.1)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.5.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (25.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.54->shap) (0.43
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.5)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2025
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.59.2)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.2.3)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Building wheels for collected packages: lime
  Building wheel for lime (setup.py) ... done
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834 sha256=b2b3b93fbbba43b0ff465006968c0e2920bbe0a4860b9e3
  Stored in directory: /root/.cache/pip/wheels/e7/5d/0e/4b4fff9a47468fed5633211fb3b76d1db43fe806a17fb7486a
Successfully built lime
Installing collected packages: lime
Successfully installed lime-0.2.0.1
```

```
from google.colab import files
files.upload()  # Upload kaggle.json
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipython-input-957456790.py in <cell line: 0>()
      1 from google.colab import files
----> 2 files.upload()  # Upload kaggle.json

 3 frames
/usr/local/lib/python3.12/dist-packages/google/colab/_message.py in read_reply_from_input(message_id, timeout_sec)
     94     reply = _read_next_input_message()
     95     if reply == _NOT_READY or not isinstance(reply, dict):
---> 96       time.sleep(0.025)
     97       continue
     98     if (

KeyboardInterrupt:
```

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

```
cp: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
```

```
import os
import zipfile

# Set project directory in Google Drive
project_dir = '/content/drive/MyDrive/TB_Project'
os.makedirs(project_dir, exist_ok=True)
```

```
import os
import zipfile

# Set project directory in Google Drive
project_dir = '/content/drive/MyDrive/TB_Project'
os.makedirs(project_dir, exist_ok=True)

# ✅ Download Datasets
!kaggle datasets download -d tawsifurrahman/tuberculosis-tb-chest-xray-dataset -p {project_dir}
!kaggle datasets download -d victorcaelina/tuberculosis-symptoms -p {project_dir}

# ✅ Unzip Chest X-ray Dataset
with zipfile.ZipFile(f"{project_dir}/tuberculosis-tb-chest-xray-dataset.zip", 'r') as zip_ref:
    zip_ref.extractall(f"{project_dir}/xray_data")

# ✅ Unzip Symptom Dataset
with zipfile.ZipFile(f"{project_dir}/tuberculosis-symptoms.zip", 'r') as zip_ref:
    zip_ref.extractall(f"{project_dir}/symptom_data")
```

```
# List files in symptom_data
!ls "{project_dir}/symptom_data"
```

```
'Tb disease symptoms.csv'
'Tb disease symptoms.json'
'Tb disease symptoms without id and datetime.csv'
'Tb disease symptoms without id and datetime.json'
'Tb disease symptoms.xlsx'
'Tb disease without id and datetime.xlsx'
```

```
import pandas as pd

# Use exact file name with correct folder
symptom_path = "/content/drive/MyDrive/TB_Project/symptom_data/Tb disease symptoms.csv"

# Load the dataset
symptom_df = pd.read_csv(symptom_path)

# Show basic info
print(symptom_df.columns)
```

```
symptom_df.head()
```

```
Index(['no', 'id', 'name', 'gender', 'date', 'time', 'fever for two weeks',
       'coughing blood', 'sputum mixed with blood', 'night sweats ',
       'chest pain', 'back pain in certain parts ', 'shortness of breath',
       'weight loss ', 'body feels tired',
       'lumps that appear around the armpits and neck',
       'cough and phlegm continuously for two weeks to four weeks',
       'swollen lymph nodes', 'loss of appetite'],
      dtype='object')
```

	no	id	name	gender	date	time	fever for two weeks	coughing blood	sputum mixed with blood	night sweats	chest pain	back pain in certain parts	shortness of breath	weight loss	body feels tired	lump tha appea aroun th armpit an nec
0	1	8048761033	Noe	Male	12/10/2020	4:51 PM	0	1	1	1	1	1	1	1	0	
1	2	793846900	Genna	Male	11/16/2020	9:35 AM	1	1	1	0	0	1	1	0	0	
2	3	5619727459	Leesa	Male	1/18/2020	8:38 PM	0	0	1	1	0	1	0	0	0	
3	4	4337104062	Case	Female	2/4/2020	3:09 PM	0	1	0	1	1	0	0	1	1	

```
# Strip and replace column names
symptom_df.columns = symptom_df.columns.str.strip().str.replace(' ', '_').str.lower()
```

```
#Load and Preprocess Symptom Data

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Step 1: Load dataset
symptom_df = pd.read_csv("/content/drive/MyDrive/TB_Project/symptom_data/Tb disease symptoms.csv")

# Step 2: Drop irrelevant columns
drop_cols = ['no', 'id', 'name', 'gender', 'date', 'time']
symptom_df = symptom_df.drop(columns=drop_cols)

# ✅ Step 3: Clean column names (before doing anything else)
symptom_df.columns = symptom_df.columns.str.strip().str.replace(' ', '_').str.lower()

# Step 4: Assign TB label = 1
symptom_df['label'] = 1

# Step 5: Simulate 20 non-TB records with all 0s
non_tb = pd.DataFrame(np.zeros((20, symptom_df.shape[1])), columns=symptom_df.columns)
non_tb['label'] = 0

# Step 6: Combine the TB and non-TB data
data = pd.concat([symptom_df, non_tb], ignore_index=True)

# Step 7: Split into features and target
X = data.drop(columns=['label'])
y = data['label']

# ✅ Print column names once to confirm they're clean
print(f"Columns used for training:\n{list(X.columns)}")

# Step 8: Train-test split and scale
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train).astype('float32')
X_test_scaled = scaler.transform(X_test).astype('float32')

# Step 9: Define and train the MLP model
```

```
mlp_model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])
mlp_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
mlp_model.fit(X_train_scaled, y_train, epochs=10, validation_data=(X_test_scaled, y_test))
```

```
Columns used for training:
['fever_for_two_weeks', 'coughing_blood', 'sputum_mixed_with_blood', 'night_sweats', 'chest_pain', 'back_pain_in_certain_parts', 's
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` ar
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 9s 194ms/step - accuracy: 0.7174 - loss: 0.5727 - val_accuracy: 0.9804 - val_loss: 0.2101
Epoch 2/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.9848 - loss: 0.1578 - val_accuracy: 0.9804 - val_loss: 0.0879
Epoch 3/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.9849 - loss: 0.0689 - val_accuracy: 0.9804 - val_loss: 0.0506
Epoch 4/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.9794 - loss: 0.0449 - val_accuracy: 0.9804 - val_loss: 0.0327
Epoch 5/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 0.9900 - loss: 0.0316 - val_accuracy: 1.0000 - val_loss: 0.0223
Epoch 6/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0223 - val_accuracy: 1.0000 - val_loss: 0.0159
Epoch 7/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0145 - val_accuracy: 1.0000 - val_loss: 0.0118
Epoch 8/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0107 - val_accuracy: 1.0000 - val_loss: 0.0089
Epoch 9/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0083 - val_accuracy: 1.0000 - val_loss: 0.0069
Epoch 10/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 1.0000 - loss: 0.0054 - val_accuracy: 1.0000 - val_loss: 0.0055
<keras.src.callbacks.history.History at 0x7e4f545b4860>
```

```
# List files in your TB image folder
!ls "/content/drive/MyDrive/TB_Project/xray_data/TB_Chest_Radiography_Database/Tuberculosis"
```

```
Tuberculosis-228.png  Tuberculosis-439.png  Tuberculosis-64.png
```

```
Tuberculosis-229.png  Tuberculosis-43.png   Tuberculosis-650.png
Tuberculosis-22.png   Tuberculosis-440.png  Tuberculosis-651.png
Tuberculosis-230.png  Tuberculosis-441.png  Tuberculosis-652.png
Tuberculosis-231.png  Tuberculosis-442.png  Tuberculosis-653.png
Tuberculosis-232.png  Tuberculosis-443.png  Tuberculosis-654.png
Tuberculosis-233.png  Tuberculosis-444.png  Tuberculosis-655.png
Tuberculosis-234.png  Tuberculosis-445.png  Tuberculosis-656.png
Tuberculosis-235.png  Tuberculosis-446.png  Tuberculosis-657.png
Tuberculosis-236.png  Tuberculosis-447.png  Tuberculosis-658.png
Tuberculosis-237.png  Tuberculosis-448.png  Tuberculosis-659.png
Tuberculosis-238.png  Tuberculosis-449.png  Tuberculosis-65.png
Tuberculosis-239.png  Tuberculosis-44.png   Tuberculosis-660.png
Tuberculosis-23.png   Tuberculosis-450.png  Tuberculosis-661.png
Tuberculosis-240.png  Tuberculosis-451.png  Tuberculosis-662.png
Tuberculosis-241.png  Tuberculosis-452.png  Tuberculosis-663.png
```

```
# Scale and Train MLP on Symptoms
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train).astype('float32')
X_test_scaled = scaler.transform(X_test).astype('float32')

mlp_model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])
mlp_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
mlp_model.fit(X_train_scaled, y_train, epochs=10, validation_data=(X_test_scaled, y_test))
```

```
Epoch 1/10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` ar
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
26/26 ━━━━━━━━━━━━━━━━━━━━ 3s 53ms/step - accuracy: 0.7876 - loss: 0.5427 - val_accuracy: 0.9804 - val_loss: 0.2330
Epoch 2/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9802 - loss: 0.1771 - val_accuracy: 0.9804 - val_loss: 0.0910
Epoch 3/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9724 - loss: 0.0901 - val_accuracy: 0.9804 - val_loss: 0.0521
Epoch 4/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9833 - loss: 0.0409 - val_accuracy: 0.9804 - val_loss: 0.0344
Epoch 5/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9804 - loss: 0.0307 - val_accuracy: 1.0000 - val_loss: 0.0247
Epoch 6/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 1.0000 - loss: 0.0200 - val_accuracy: 1.0000 - val_loss: 0.0191
Epoch 7/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 1.0000 - loss: 0.0168 - val_accuracy: 1.0000 - val_loss: 0.0149
Epoch 8/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss: 0.0154 - val_accuracy: 1.0000 - val_loss: 0.0122
Epoch 9/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss: 0.0105 - val_accuracy: 1.0000 - val_loss: 0.0102
Epoch 10/10
26/26 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 1.0000 - loss: 0.0095 - val_accuracy: 1.0000 - val_loss: 0.0086
<keras.src.callbacks.history.History at 0x7e4f501e3aa0>
```

```
import os
import random
import numpy as np
import pandas as pd
from PIL import Image
import matplotlib.pyplot as plt

import torch
import torchvision
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import Dataset
from torchvision.io import read_image
from torch.utils.data import DataLoader
from torchvision import datasets, models, transforms
```

```
# Initital configurations
SEED = 1234

def set_seeds(seed=1234):
    """Set seeds for reproducibility."""
    np.random.seed(seed)
    random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed) # multi-GPU

# Set seeds for reproducibility
set_seeds(seed=SEED)

# Set device
cuda = True
device = torch.device("cuda" if (
    torch.cuda.is_available() and cuda) else "cpu")
```

```
from torchvision.models import densenet121,DenseNet121_Weights, DenseNet

#model architecture
class CXR_DenseNetModel(nn.Module):
  """DenseNet121 pretrained model definition."""
  def __init__(self, num_classes):
      super(CXR_DenseNetModel, self).__init__()

      self.model = torchvision.models.densenet121(pretrained=True)

      # Freeze the model's parameters
      for param in self.model.parameters():
          param.requires_grad = False

      # Replace the last linear layer of the model
      in_features = self.model.classifier.in_features
      self.model.classifier = nn.Sequential(
          nn.Linear(in_features, in_features // 2),
          nn.Dropout(0.5),
          nn.Linear(in_features // 2, in_features // 4),
          # nn.Dropout(0.5),
          nn.Linear(in_features // 4, in_features // 8),
          nn.Dropout(0.25),
          nn.Linear(in_features // 8, num_classes),
      )

  def forward(self, x):
      return self.model(x)

model = CXR_DenseNetModel(num_classes=2)


# set device
model = model.to(device)
model
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated s
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `Non
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/densenet121-a639ec97.pth" to /root/.cache/torch/hub/checkpoints/densenet121-a63
100%|██████████| 30.8M/30.8M [00:00<00:00, 126MB/s]
CXR_DenseNetModel(
  (model): DenseNet(
    (features): Sequential(
      (conv0): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
      (norm0): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu0): ReLU(inplace=True)
      (pool0): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
      (denseblock1): _DenseBlock(
        (denselayer1): _DenseLayer(
          (norm1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu1): ReLU(inplace=True)
          (conv1): Conv2d(64, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (norm2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu2): ReLU(inplace=True)
          (conv2): Conv2d(128, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
```

```
        )
        (denselayer2): _DenseLayer(
          (norm1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu1): ReLU(inplace=True)
          (conv1): Conv2d(96, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (norm2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu2): ReLU(inplace=True)
          (conv2): Conv2d(128, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        )
        (denselayer3): _DenseLayer(
          (norm1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu1): ReLU(inplace=True)
          (conv1): Conv2d(128, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (norm2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu2): ReLU(inplace=True)
          (conv2): Conv2d(128, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        )
        (denselayer4): _DenseLayer(
          (norm1): BatchNorm2d(160, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu1): ReLU(inplace=True)
          (conv1): Conv2d(160, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (norm2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu2): ReLU(inplace=True)
          (conv2): Conv2d(128, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        )
        (denselayer5): _DenseLayer(
          (norm1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu1): ReLU(inplace=True)
          (conv1): Conv2d(192, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (norm2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu2): ReLU(inplace=True)
          (conv2): Conv2d(128, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        )
        (denselayer6): _DenseLayer(
          (norm1): BatchNorm2d(224, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu1): ReLU(inplace=True)
```

```
!pip install onnx
```

```
Collecting onnx
  Downloading onnx-1.19.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (7.0 kB)
Requirement already satisfied: numpy>=1.22 in /usr/local/lib/python3.12/dist-packages (from onnx) (2.0.2)
Requirement already satisfied: protobuf>=4.25.1 in /usr/local/lib/python3.12/dist-packages (from onnx) (5.29.5)
Requirement already satisfied: typing_extensions>=4.7.1 in /usr/local/lib/python3.12/dist-packages (from onnx) (4.15.0)
Requirement already satisfied: ml_dtypes in /usr/local/lib/python3.12/dist-packages (from onnx) (0.5.3)
Downloading onnx-1.19.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (18.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 18.2/18.2 MB 70.4 MB/s eta 0:00:00
Installing collected packages: onnx
Successfully installed onnx-1.19.0
```

```
sample = pd.DataFrame([[
    1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0  # Match your 13 features
]], columns=X.columns)

sample_scaled = scaler.transform(sample).astype('float32')
```

```
import torchvision.transforms as transforms

def preprocess_image(image_path):
    img = Image.open(image_path).convert('RGB')  # Ensure 3 channels
    transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                             [0.229, 0.224, 0.225])
    ])
    img_tensor = transform(img).unsqueeze(0)  # Shape: (1, 3, 224, 224)
    return img_tensor
```

```
!pip install onnxruntime
```

```
Collecting onnxruntime
  Downloading onnxruntime-1.22.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (4.9 kB)
Collecting coloredlogs (from onnxruntime)
  Downloading coloredlogs-15.0.1-py2.py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: flatbuffers in /usr/local/lib/python3.12/dist-packages (from onnxruntime) (25.2.10)
```

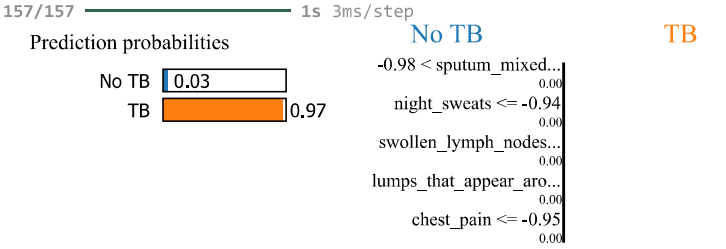
```
Requirement already satisfied: numpy>=1.21.6 in /usr/local/lib/python3.12/dist-packages (from onnxruntime) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from onnxruntime) (25.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from onnxruntime) (5.29.5)
Requirement already satisfied: sympy in /usr/local/lib/python3.12/dist-packages (from onnxruntime) (1.13.3)
Collecting humanfriendly>=9.1 (from coloredlogs->onnxruntime)
  Downloading humanfriendly-10.0-py2.py3-none-any.whl.metadata (9.2 kB)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy->onnxruntime) (1.3.0)
Downloading onnxruntime-1.22.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.5/16.5 MB 125.0 MB/s eta 0:00:00
Downloading coloredlogs-15.0.1-py2.py3-none-any.whl (46 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.0/46.0 kB 4.2 MB/s eta 0:00:00
Downloading humanfriendly-10.0-py2.py3-none-any.whl (86 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 86.8/86.8 kB 9.4 MB/s eta 0:00:00
Installing collected packages: humanfriendly, coloredlogs, onnxruntime
Successfully installed coloredlogs-15.0.1 humanfriendly-10.0 onnxruntime-1.22.1
```

```
# Inference from image model (ONNX)
import onnxruntime as ort
import torch.nn.functional as F

# Define a placeholder for xray_path and preprocess the image
xray_path = "/content/drive/MyDrive/TB_Project/xray_data/TB_Chest_Radiography_Database/Tuberculosis/Tuberculosis-1.png" # Placehol
image_tensor = preprocess_image(xray_path)

ort_session = ort.InferenceSession("tuberModel.onnx")
ort_inputs = {ort_session.get_inputs()[0].name: image_tensor.numpy()}
ort_outs = ort_session.run(None, ort_inputs)

logits = ort_outs[0][0]
probs = F.softmax(torch.tensor(logits), dim=0).numpy()

img_pred = probs[1]

# Inference from symptom model (Keras)
symptom_pred = mlp_model.predict(sample_scaled)[0][0]

# Combine predictions
final_score = (img_pred + symptom_pred) / 2
final_label = 1 if final_score > 0.5 else 0

print(f"Image Model: {img_pred:.3f}, Symptom Model: {symptom_pred:.3f}, Final: {final_score:.3f} → Label: {'TB' if final_label els
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 220ms/step
Image Model: 1.000, Symptom Model: 0.967, Final: 0.984 → Label: TB
```

```
import shap
import matplotlib.pyplot as plt
from lime.lime_tabular import LimeTabularExplainer
import cv2
import numpy as np
import pandas as pd
from IPython.display import display, HTML

def mlp_predict_proba(x):
    p = mlp_model.predict(x)
    return np.concatenate([1 - p, p], axis=1)  # shape: (n_samples, 2)


lime_explainer = LimeTabularExplainer(
    X_train_scaled,
    feature_names=X.columns.tolist(),
    class_names=["No TB", "TB"],
    discretize_continuous=True
)

lime_exp_sym = lime_explainer.explain_instance(
    sample_scaled[0],
    mlp_predict_proba,
    num_features=5
)
lime_exp_sym.show_in_notebook()
```

```
157/157 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
```

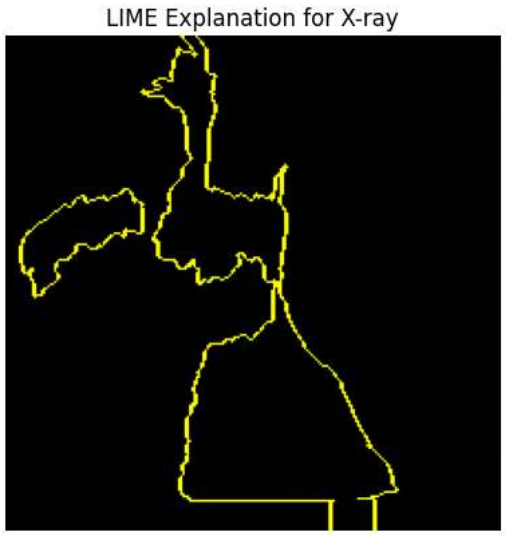



```
from torchvision import datasets, models, transforms
import torchvision.transforms as transforms
from PIL import Image

transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])
```

```
from lime import lime_image
from skimage.segmentation import mark_boundaries
import torchvision.transforms as transforms
from PIL import Image
import torch
import numpy as np
import torch.nn.functional as F
import onnxruntime as ort

# Define the transform
transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])

lime_img_explainer = lime_image.LimeImageExplainer()

# Load the ONNX session
ort_session = ort.InferenceSession("tuberModel.onnx")


def predict_image(imgs):
    # LIME will send batches of shape (H, W, 3)
    # Process each image individually due to ONNX model expecting batch size 1
    probs_list = []
    for img in imgs:
        img_tensor = transform(Image.fromarray(img.astype(np.uint8))).unsqueeze(0) # Add batch dimension
        ort_inputs = {ort_session.get_inputs()[0].name: img_tensor.numpy()}
        logits = ort_session.run(None, ort_inputs)[0][0]
        probs = F.softmax(torch.tensor(logits), dim=0).numpy()
        probs_list.append(probs)
    return np.array(probs_list)


# Define a placeholder for xray_path and preprocess the image
xray_path = "/content/drive/MyDrive/TB_Project/xray_data/TB_Chest_Radiography_Database/Tuberculosis/Tuberculosis-1.png" # Placehol
image_tensor = preprocess_image(xray_path)


explanation = lime_img_explainer.explain_instance(
    image_tensor.squeeze().permute(1, 2, 0).numpy(),
    predict_image,
    top_labels=2,
    hide_color=0,
    num_samples=1000
)

temp, mask = explanation.get_image_and_mask(
    label=1, positive_only=True, hide_rest=False
)
```

```
plt.imshow(mark_boundaries(temp / 255.0, mask))
plt.title("LIME Explanation for X-ray")
plt.axis("off")
plt.show()
```

```
100%                                                1000/1000 [01:23<00:00, 12.48it/s]
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integer
```




```
!pip install segment_anything
```

```
Collecting segment_anything
  Downloading segment_anything-1.0-py3-none-any.whl.metadata (487 bytes)
Downloading segment_anything-1.0-py3-none-any.whl (36 kB)
Installing collected packages: segment_anything
Successfully installed segment_anything-1.0
```

```
"""
TB detection: Segmentation-aware SHAP + hierarchical LIME prototype

This single-file prototype demonstrates a runnable pipeline you can run locally.
It supports two modes:
  - DEMO mode: uses a placeholder image & mock predictors so you can see visualizations immediately.
  - REAL mode: plugs in your ONNX image model (DenseNet) and your trained MLP for symptoms.

Features implemented:
  - Automatic fallback segmentation: tries SAM (if installed) else skimage SLIC superpixels.
  - Region-level SHAP (Kernel SHAP) treating each segment as a feature and allowing tabular features.
  - LIME with a segmentation function derived from SAM/SLIC.
  - CLIP-based region naming if CLIP is installed; else simple intensity-based labels.
  - Visualizations: SHAP region overlay, LIME boundaries, top concepts and symptom importances.

Notes:
  - For publication-grade results use SAM + a medical V-L model + real ONNX/MLP and radiologist masks.
  - Kernel SHAP can be slow; keep number of segments <= 15 for interactive runs.

Requirements (pip):
  pip install numpy matplotlib scikit-image shap lime opencv-python pillow torch torchvision onnxruntime
  # optional (better results):
  pip install git+https://github.com/facebookresearch/segment-anything git+https://github.com/openai/CLIP.git

Replace the PATHS below with your own ONNX model / MLP files.

"""

# %% imports
import os
import sys
import time
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from skimage import data, color, img_as_ubyte
from skimage.segmentation import slic
```

```
from skimage.util import img_as_float
import cv2
import shap
from lime import lime_image
from skimage.segmentation import mark_boundaries

# optional imports (wrapped in try/except)
try:
    import torch
    import torchvision.transforms as T
except Exception:
    torch = None

try:
    import onnxruntime as ort
except Exception:
    ort = None

try:
    # segment-anything
    from segment_anything import SamPredictor, sam_model_registry, SamAutomaticMaskGenerator
    SAM_AVAILABLE = True
except Exception:
    SAM_AVAILABLE = False

try:
    import clip
    CLIP_AVAILABLE = True
except Exception:
    CLIP_AVAILABLE = False

# %% config - change these to point to your models/data
CONFIG = {
    "MODE": "DEMO",  # or "REAL"
    "ONNX_PATH": "path/to/your/image_model.onnx",
    "IMAGE_PATH": "/content/drive/MyDrive/TB_Project/xray_data/TB_Chest_Radiography_Database/Tuberculosis/Tuberculosis-1.png",  #
    "DEVICE": "cuda" if (torch is not None and torch.cuda.is_available()) else "cpu",
    "MAX_SEGMENTS": 12,
    "SHAP_NSAMPLES": 256,
}

# %% utility functions

def load_demo_image():
    # use skimage camera as a placeholder; convert to RGB
    im = data.camera()
    im = img_as_ubyte(im)
    pil = Image.fromarray(im).convert("RGB")
    return pil

# simple show helper
def show_images_grid(images, titles=None, figsize=(12,6)):
    n = len(images)
    cols = min(n, 3)
    rows = (n + cols - 1) // cols
    plt.figure(figsize=figsize)
    for i, im in enumerate(images):
        plt.subplot(rows, cols, i+1)
        if isinstance(im, np.ndarray):
            plt.imshow(im)
        else:
            plt.imshow(np.array(im))
        plt.axis('off')
        if titles and i < len(titles):
            plt.title(titles[i])
    plt.show()

# %% model wrappers (DEMO and placeholders)

# DEMO image predictor: simple brightness-based heuristic
def demo_predict_image(imgs):
    # imgs: list of HxWx3 uint8
    probs = []
    for img in imgs:
        gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
        mean = gray.mean() / 255.0
        # make up a TB probability heuristic: darker chest center -> higher "TB"
```

```
            p_tb = float(max(0.05, min(0.95, 1 - mean)))
            probs.append([1-p_tb, p_tb])
        return np.array(probs)

    # DEMO MLP: random but reproducible
    def demo_mlp_proba(x):
        rng = np.random.RandomState(0)
        n = x.shape[0]
        p = rng.beta(2,5,size=(n,))  # arbitrary
        return np.vstack([1-p, p]).T

    # REAL wrappers (user must provide models)
    ort_session = None
    if CONFIG['MODE'] == 'REAL' and ort is not None and os.path.exists(CONFIG['ONNX_PATH']):
        ort_session = ort.InferenceSession(CONFIG['ONNX_PATH'])

    # user should define transforms consistent with training
    if torch is not None:
        default_transform = T.Compose([
            T.Resize((224,224)),
            T.ToTensor(),
            T.Normalize(mean=[0.485,0.485,0.485], std=[0.229,0.229,0.229])
        ])
    else:
        default_transform = None


    def predict_image_wrapper(imgs):
        if CONFIG['MODE'] == 'DEMO' or ort_session is None:
            return demo_predict_image(imgs)
        # else run ONNX
        batch = []
        for img in imgs:
            pil = Image.fromarray(img.astype(np.uint8))
            if pil.mode != 'RGB':
                pil = pil.convert('RGB')
            if default_transform is not None:
                t = default_transform(pil)
                arr = t.numpy()
            else:
                arr = np.array(pil).transpose(2,0,1).astype(np.float32) / 255.0
            batch.append(arr)
        batch_np = np.stack(batch, axis=0).astype(np.float32)
        # ONNX input may vary; here we guess the first input
        inp_name = ort_session.get_inputs()[0].name
        logits = ort_session.run(None, {inp_name: batch_np})[0]
        import torch as _torch
        probs = _torch.nn.functional.softmax(_torch.tensor(logits), dim=1).numpy()
        return probs


    def mlp_predict_proba_wrapper(x):
        if CONFIG['MODE'] == 'DEMO':
            return demo_mlp_proba(x)
        # else try to load scikit-learn model from pickle
        import joblib
        if not os.path.exists(CONFIG['MLP_PATH']):
            raise FileNotFoundError('MLP_PATH not found for REAL mode')
        model = joblib.load(CONFIG['MLP_PATH'])
        p = model.predict_proba(x)
        return p

    # alias names expected by the earlier discussion
    predict_image = predict_image_wrapper
    mlp_predict_proba = mlp_predict_proba_wrapper

    # %% segmentation utilities

    def slic_segmentation(pil_image, n_segments=12, compactness=10):
        arr = np.array(pil_image)
        arr_float = img_as_float(arr)
        seg = slic(arr_float, n_segments=n_segments, compactness=compactness, start_label=0)
        # build boolean masks list
        masks = []
        for label in np.unique(seg):
            masks.append(seg == label)
        return masks, seg
```

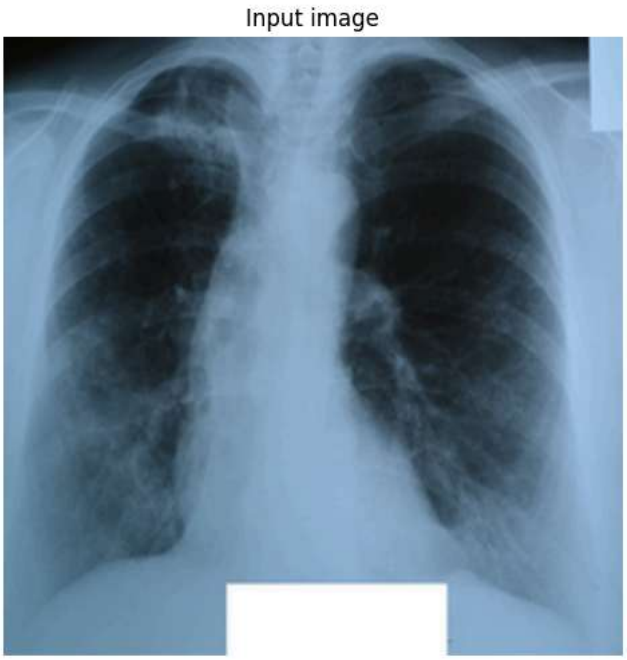
```
# SAM-based segmentation (if SAM_AVAILABLE)
def sam_segment_image(predictor, pil_image, min_area=500):
    im_np = np.array(pil_image)
    mask_generator = SamAutomaticMaskGenerator(predictor.model)
    masks_info = mask_generator.generate(im_np)
    masks = []
    boxes = []
    for mi in masks_info:
        mask = mi['segmentation']
        if mask.sum() < min_area:
            continue
        masks.append(mask.astype(bool))
        boxes.append(mi.get('bbox', None))
    return masks, boxes


def get_masks_for_image(pil_image, max_masks=12, sam_predictor=None):
    if SAM_AVAILABLE and sam_predictor is not None:
        masks, boxes = sam_segment_image(sam_predictor, pil_image)
    else:
        masks, seg = slic_segmentation(pil_image, n_segments=max_masks)
    # sort masks by area desc and keep top-k
    masks = sorted(masks, key=lambda m: m.sum(), reverse=True)
    masks = masks[:max_masks]
    return masks

# %% CLIP-based region naming (fallback: intensity labels)

def name_masks_with_clip_fallback(pil_image, masks, candidate_labels=None):
    if CLIP_AVAILABLE:
        device = CONFIG['DEVICE'] if torch is not None else 'cpu'
        model, preprocess = clip.load('ViT-B/32', device=device)
        if candidate_labels is None:
            candidate_labels = [
                'consolidation','cavity','pleural effusion','atelectasis','normal lung',
                'infiltrate','cardiomegaly','tuberculosis','nodule','opacity','fibrosis','artifact'
            ]
        text_tokens = clip.tokenize(candidate_labels).to(device)
        with torch.no_grad():
            text_feats = model.encode_text(text_tokens)
            text_feats = text_feats / text_feats.norm(dim=-1, keepdim=True)
        labels = []
        scores = []
        im_np = np.array(pil_image)
        for mask in masks:
            ys, xs = np.where(mask)
            if len(xs) == 0:
                labels.append('unknown'); scores.append(0.0); continue
            y0, y1 = ys.min(), ys.max()
            x0, x1 = xs.min(), xs.max()
            crop = Image.fromarray(im_np[y0:y1+1, x0:x1+1]).convert('RGB')
            image_input = preprocess(crop).unsqueeze(0).to(device)
            with torch.no_grad():
                img_feat = model.encode_image(image_input)
                img_feat = img_feat / img_feat.norm(dim=-1, keepdim=True)
                sims = (img_feat @ text_feats.T).squeeze(0).cpu().numpy()
            best_idx = sims.argmax()
            labels.append(candidate_labels[best_idx])
            scores.append(float(sims[best_idx]))
        return labels, scores
    else:
        # fallback: simple label by mean intensity
        labels = []
        scores = []
        im_np = np.array(pil_image.convert('L'))
        for i, mask in enumerate(masks):
            avg = im_np[mask].mean()
            if avg < 80:
                labels.append('dense_region')
            elif avg < 160:
                labels.append('mid_density')
            else:
                labels.append('bright_region')
            scores.append(float(avg))
        return labels, scores
```

```
# %% masking and fusion

def apply_region_mask_to_image(img_np, masks, region_bits, mask_mode='blur'):
    out = img_np.copy().astype(np.float32)
    H, W = out.shape[:2]
    for bit, mask in zip(region_bits, masks):
        if int(round(bit)) == 1:
            continue
        if mask_mode == 'black':
            out[mask] = 0
        elif mask_mode == 'mean':
            mean_val = out.mean(axis=(0,1))
            out[mask] = mean_val
        elif mask_mode == 'blur':
            ys, xs = np.where(mask)
            if len(xs) == 0:
                continue
            y0, y1 = ys.min(), ys.max()
            x0, x1 = xs.min(), xs.max()
            # clip to image
            y0, y1 = max(0,y0), min(H-1,y1)
            x0, x1 = max(0,x0), min(W-1,x1)
            patch = out[y0:y1+1, x0:x1+1].astype(np.uint8)
            if patch.size == 0:
                continue
            k = max(3, (min(patch.shape[0], patch.shape[1])//10)|1)
            k = k if k%2==1 else k+1
            patch_blur = cv2.GaussianBlur(patch, (k,k), 0)
            submask = mask[y0:y1+1, x0:x1+1]
            out[y0:y1+1, x0:x1+1][submask] = patch_blur[submask]
    return out.astype(np.uint8)


def fused_predict_proba_from_image_and_tabular(img_np, tabular_vector, image_weight=0.7):
    probs_image = predict_image([img_np])[0]
    probs_tab = mlp_predict_proba(tabular_vector.reshape(1,-1))[0]
    fused = image_weight * probs_image + (1-image_weight) * probs_tab
    return fused

# %% SHAP wrapper

def shap_model_from_z(z, background_img_np, masks, tabular_mean, image_weight=0.7):
    k = len(masks)
    region_bits = np.round(z[:k]).astype(int)
    tabular_vals = z[k:]
    masked_img = apply_region_mask_to_image(background_img_np, masks, region_bits, mask_mode='blur')
    probs = fused_predict_proba_from_image_and_tabular(masked_img, tabular_vals, image_weight=image_weight)
    return float(probs[1])

# %% LIME segmentation wrapper

def sam_like_segmentation_fn(img_arr, max_masks=12):
    pil = Image.fromarray(img_arr.astype(np.uint8)).convert('RGB')
    masks = get_masks_for_image(pil, max_masks=max_masks)
    H, W = img_arr.shape[:2]
    seg = np.zeros((H,W), dtype=int) - 1
    for i, m in enumerate(masks):
        seg[m] = i
    # fill leftover
    if (seg == -1).any():
        seg[seg==-1] = seg.max()+1
    return seg

# %% visualization helpers

def plot_region_shap_overlay(image_np, masks, shap_vals_region, labels):
    overlay = image_np.copy().astype(float)
    vals = np.array(shap_vals_region)
    # normalize
    if vals.max() - vals.min() < 1e-8:
        norm = (vals - vals.min())
    else:
        norm = (vals - vals.min()) / (vals.max() - vals.min())
    for i, m in enumerate(masks):
        c = norm[i]
        # red tint
        overlay[m] = overlay[m] * (1 - 0.6*c) + np.array([255, 0, 0]) * (0.6*c)
```

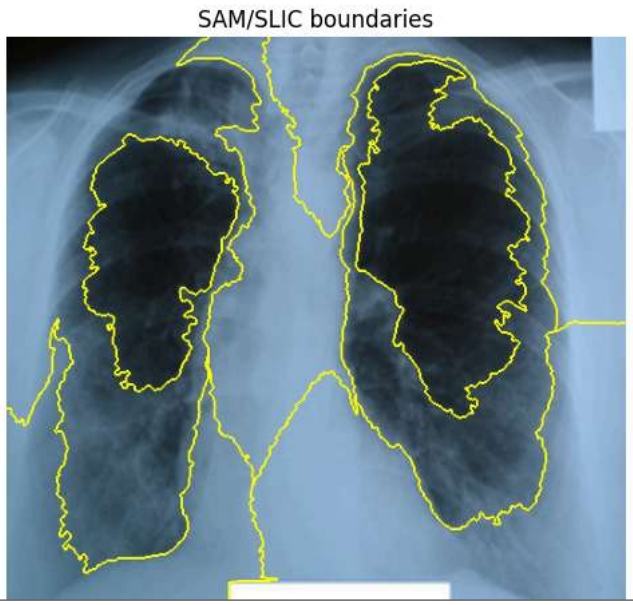
```
        plt.figure(figsize=(6,6))
        plt.imshow(overlay.astype(np.uint8))
        plt.axis('off')
        # text
        for i, lab in enumerate(labels[:10]):
            plt.text(5, 12 + 12*i, f"{lab}: {shap_vals_region[i]:.3f}", color='white', fontsize=9, backgroundcolor='black')
        plt.title('Region-level SHAP overlay')
        plt.show()

    # %% end-to-end demo run

    def run_demo():
        print('Running demo pipeline...')
        pil = load_demo_image() if CONFIG['IMAGE_PATH'] is None else Image.open(CONFIG['IMAGE_PATH']).convert('RGB')
        image_np = np.array(pil)
        show_images_grid([pil], ['Input image'])

        masks = get_masks_for_image(pil, max_masks=CONFIG['MAX_SEGMENTS'])
        labels, scores = name_masks_with_clip_fallback(pil, masks)
        print('Found', len(masks), 'masks. Labels:', labels)

        # display boundaries
        temp = image_np.copy()
        # create a label map for mark_boundaries (use integer label map)
        H,W = image_np.shape[:2]
        segmap = np.zeros((H,W), dtype=int)
        for i,m in enumerate(masks):
            segmap[m] = i+1
        bound = mark_boundaries(image_np/255.0, segmap)
        show_images_grid([bound], ['SAM/SLIC boundaries'])

        # build background/tabular placeholder
        tabular_mean = np.zeros((5,))  # DEMO; replace with real feature mean and sample
        sample_tab = np.array([0.1, 0.0, 1.0, 0.5, 0.2])

        # build shap background and explainer
        k = len(masks)
        background = np.concatenate([np.zeros((1,k)), tabular_mean.reshape(1,-1)], axis=1)
        # lambda wrapper for shap
        def shap_fn(z_batch):
            out = []
            for z in z_batch:
                out.append(shap_model_from_z(z, image_np, masks, tabular_mean))
            return np.array(out)

        explainer = shap.KernelExplainer(lambda z: shap_fn(z), background)
        z0 = np.concatenate([np.ones(k), sample_tab])
        print('Computing SHAP values (nsamples=%d) ...' % CONFIG['SHAP_NSAMPLES'])
        shap_vals = explainer.shap_values(z0, nsamples=CONFIG['SHAP_NSAMPLES'])
        # shap_vals shape may be (1,) or array; try to coerce
        try:
            region_shap = np.array(shap_vals)
            # if shap returns list for classes, take class 1
            if isinstance(region_shap, list):
                region_shap = np.array(region_shap[0])
        except Exception:
            region_shap = np.zeros(k)
        # region part
        region_shap_vals = region_shap[:k]

        plot_region_shap_overlay(image_np, masks, region_shap_vals, labels)

        # LIME explanation
        print('Computing LIME explanation...')
        lime_explainer = lime_image.LimeImageExplainer()
        seg_fn = lambda x: sam_like_segmentation_fn(x, max_masks=CONFIG['MAX_SEGMENTS'])
        explanation = lime_explainer.explain_instance(
            image_np,
            classifier_fn=lambda imgs: predict_image([img_as_ubyte(i) if i.max()<=1.0 else i for i in imgs]),
            top_labels=2,
            hide_color=0,
            num_samples=256,
            segmentation_fn=seg_fn
        )
        temp, mask = explanation.get_image_and_mask(label=1, positive_only=True, hide_rest=False, num_features=5)
        plt.figure(figsize=(6,6))
        plt.imshow(mark_boundaries(temp/255.0, mask))
```

```
    plt.axis('off')
    plt.title('LIME positive regions for class 1')
    plt.show()

    # show top concepts and mock symptom importances
    print('\nTop region concepts:')
    for i,(lab, sc) in enumerate(zip(labels, scores)):
        print(f'{i+1}. {lab} (score {sc:.3f})')
    print('\nTop symptom importances (DEMO):')
    demo_symptom_names = ['cough','fever','weight_loss','night_sweats','hemoptysis']
    demo_sym_imp = np.linspace(0.5, 0.1, len(demo_symptom_names))
    for n,v in zip(demo_symptom_names, demo_sym_imp):
        print(f'{n}: {v:.3f}')

    print('\nDemo pipeline finished.')

# %% run
if __name__ == '__main__':
    run_demo()
```

```
Running demo pipeline...
```




```
Found 8 masks. Labels: ['bright_region', 'mid_density', 'mid_density', 'mid_density', 'dense_region', 'mid_density', 'mid_density
```




```
# Dependencies: numpy, cv2, skimage, sklearn (for auc), optional: segment-anything, lime, shap
import numpy as np
import cv2
```

```
# ---------- 1) Segmentation / masks ----------
def get_segments_or_masks(image_np, method="slic", n_segments=200, compactness=10, sam_model=None, sam_checkpoint=None, sam_model_
    """
    Returns:
      segments: (H,W) int labels (0 means background/unassigned)
      masks: list of boolean arrays (one per region) for convenience (masks[i] == (segments == labels[i]))
    method: "sam" or "slic"
    """
    H, W = image_np.shape[:2]
    # ensure uint8 image for sam / cv2 expectations
    img_uint8 = (np.clip(image_np, 0, 1) * 255).astype(np.uint8) if image_np.max() <= 1.0 else image_np.astype(np.uint8)

    if method.lower() == "sam":
        try:
            from segment_anything import SamAutomaticMaskGenerator, sam_model_registry
        except Exception as e:
            raise ImportError("segment-anything not available. Install it or use method='slic'") from e

        if sam_model is None:
            if sam_checkpoint is None:
                raise ValueError("Provide sam_model or sam_checkpoint to use SAM.")
            sam = sam_model_registry[sam_model_type](checkpoint=sam_checkpoint)
        else:
            sam = sam_model

        mask_generator = SamAutomaticMaskGenerator(sam)
        ann = mask_generator.generate(img_uint8)  # list of dicts with 'segmentation' boolean arrays

        segments = np.zeros((H, W), dtype=np.int32)
        for i, m in enumerate(ann, start=1):
            seg_bool = m["segmentation"].astype(bool)
            segments[seg_bool] = i   # later masks overwrite earlier overlaps
        masks = [segments == i for i in range(1, segments.max() + 1)] if segments.max() > 0 else []
        return segments, masks

    else:
        # fallback: SLIC superpixels
        from skimage import img_as_float
        from skimage.segmentation import slic
        image_float = img_as_float(img_uint8)
        segments = slic(image_float, n_segments=n_segments, compactness=compactness, start_label=1)
        masks = [segments == lab for lab in np.unique(segments) if lab != 0]
        return segments, masks


# ---------- 2) Turn any SHAP/LIME explanation into a grayscale saliency map ----------
def to_grayscale_saliency(saliency):
    """
    Accepts saliency as:
      - (H,W) -> returned as-is (normalized)
      - (H,W,3) -> mean(abs(channel)) across channels
      - (3,H,W) -> transposed to (H,W,3)
    Returns normalized saliency in [0,1]
    """
    sal = np.array(saliency)
    if sal.ndim == 3:
        if sal.shape[0] == 3 and sal.shape[2] != 3:
            sal = np.transpose(sal, (1, 2, 0))
        sal_gray = np.mean(np.abs(sal), axis=2)
    elif sal.ndim == 2:
        sal_gray = sal.astype(float)
    else:
        raise ValueError("Unexpected saliency shape: " + str(sal.shape))

    sal_gray = sal_gray - sal_gray.min()
    if sal_gray.max() > 0:
        sal_gray = sal_gray / sal_gray.max()
    return sal_gray


# ---------- 3) Map saliency -> region scores ----------
def saliency_to_region_scores(saliency, segments):
    """
    segments: (H,W) int labels (0 ignored)
    saliency: (H,W) or (H,W,3) SHAP/LIME map
    Returns:
```

```
      labels: array of labels (same order used below)
      region_scores: float array same length as labels
    """
    sal_gray = to_grayscale_saliency(saliency)
    labels = np.unique(segments)
    labels = labels[labels != 0]
    region_scores = np.array([sal_gray[segments == lab].mean() if np.any(segments == lab) else 0.0 for lab in labels])
    # Normalize region scores to [0,1] to avoid scale mismatches between SHAP and LIME
    if region_scores.max() > region_scores.min():
        region_scores = (region_scores - region_scores.min()) / (region_scores.max() - region_scores.min())
    return labels, region_scores


# ---------- 4) Batched prediction helper that supports fused image+tabular predict functions ----------
def call_predict_fn(predict_fn, images_np, sample_tab=None):
    """
    images_np: list/array of images (N,H,W,3)
    sample_tab: single tabular sample (1D) or None. If provided, it duplicates it across the batch.
    predict_fn: either:
       - predict_fn(images) -> array (N,) or (N, n_classes)
       - predict_fn(images, tabular_batch) -> array
    Returns: 1D array of probabilities (positive-class), shape (N,)
    """
    imgs = np.asarray(images_np)
    # try calling with (images, tabular) first if sample_tab provided
    try:
        if sample_tab is not None:
            tab_batch = np.repeat(np.expand_dims(np.asarray(sample_tab), 0), len(imgs), axis=0)
            preds = predict_fn(imgs, tab_batch)
        else:
            preds = predict_fn(imgs)
    except TypeError:
        # fallback - predict_fn may not accept tabular; try without it
        preds = predict_fn(imgs)

    preds = np.asarray(preds)
    if preds.ndim == 1:
        return preds
    if preds.ndim == 2:
        # assume probs over classes; return positive class if 2 columns else max-prob
        if preds.shape[1] == 2:
            return preds[:, 1]
        else:
            return preds.max(axis=1)
    # otherwise flatten
    return preds.reshape((preds.shape[0], -1)).max(axis=1)


# ---------- 5) Deletion & insertion curves ----------
def deletion_insertion_curves(image, segments, region_scores, predict_fn, sample_tab=None, steps=None, baseline_mode="blur"):
    """
    image: (H,W,3) uint8 or float
    segments: (H,W) int labels (0 ignored)
    region_scores: 1D array aligned with labels returned by saliency_to_region_scores
    predict_fn: function(images[, tab_batch]) -> probs
    sample_tab: optional tabular sample for fused predict
    steps: number of region steps to evaluate (default = number of regions)
    baseline_mode: "blur" or "mean" or callable(image)->baseline_image
    Returns: dict with deletion_preds, insertion_preds, ordered_labels, x_steps
    """
    labels = np.unique(segments)
    labels = labels[labels != 0]
    if len(labels) == 0:
        raise ValueError("No regions in segments (all zeros).")

    # Map region_scores to the same label order
    # (saliency_to_region_scores returns labels in the same unique order) - assume same order
    order_idx = np.argsort(-region_scores)  # descending importance
    ordered_labels = labels[order_idx]
    n_regions = len(ordered_labels)
    if steps is None:
        steps = n_regions

    # baseline image
    if callable(baseline_mode):
        baseline_img = baseline_mode(image)
    else:
```

```
            if baseline_mode == "mean":
                baseline_img = np.ones_like(image) * int(np.mean(image))
            else:
                # gaussian blur (works for uint8 or float)
                k = 51 if max(image.shape[:2]) > 50 else (max(image.shape[:2])//2*2+1)
                baseline_img = cv2.GaussianBlur(image.astype(np.uint8), (k, k), 0)

        # helper to apply mask union and get one modified image
        H, W = segments.shape
        # Ensure dtype consistent
        image = image.astype(baseline_img.dtype)

        # deletion: progressively replace union of top-k regions with baseline
        deletion_preds = []
        mask_union = np.zeros((H, W), dtype=bool)
        # initial (no deletion)
        deletion_preds.append(call_predict_fn(predict_fn, [image], sample_tab)[0])
        for i in range(min(steps, n_regions)):
            lab = ordered_labels[i]
            mask_union |= (segments == lab)
            img_mod = image.copy()
            img_mod[mask_union] = baseline_img[mask_union]
            deletion_preds.append(call_predict_fn(predict_fn, [img_mod], sample_tab)[0])

        # insertion: start from baseline, add regions in descending order
        insertion_preds = []
        mask_union = np.zeros((H, W), dtype=bool)
        cur_img = baseline_img.copy()
        insertion_preds.append(call_predict_fn(predict_fn, [cur_img], sample_tab)[0])
        for i in range(min(steps, n_regions)):
            lab = ordered_labels[i]
            mask_union |= (segments == lab)
            cur_img[mask_union] = image[mask_union]
            insertion_preds.append(call_predict_fn(predict_fn, [cur_img], sample_tab)[0])

        x_steps = np.arange(0, min(steps, n_regions) + 1)  # 0..k
        return {
            "deletion": np.array(deletion_preds),
            "insertion": np.array(insertion_preds),
            "ordered_labels": ordered_labels,
            "x": x_steps
        }


    # ---------- 6) Simple AUC summarizer ----------
    def summarize_curves(curves):
        """
        curves: dict returned by deletion_insertion_curves
        Returns deletion_auc, insertion_auc (normalized between 0..1)
        """
        from sklearn.metrics import auc
        x = curves["x"]
        del_vals = curves["deletion"]
        ins_vals = curves["insertion"]

        # normalize x to [0,1] so area comparable across different region counts
        x_norm = (x - x.min()) / (x.max() - x.min() + 1e-12)
        del_auc = auc(x_norm, del_vals)  # lower is better for deletion (steeper drop)
        ins_auc = auc(x_norm, ins_vals)  # higher is better for insertion (steeper rise)

        # optionally normalize by the area of a random curve if desired - here we return raw AUC
        return {"deletion_auc": del_auc, "insertion_auc": ins_auc}
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import spearmanr
from sklearn.metrics import auc

# ---------- helper: mask apply (reuse from prototype) ----------
def occlude_regions(image_np, masks, keep_indices, mode='blur'):
    """
    keep_indices: set/list of masks to keep (1), others will be occluded.
    returns occluded image.
    """
    region_bits = [1 if i in keep_indices else 0 for i in range(len(masks))]
```

```
    return apply_region_mask_to_image(image_np, masks, region_bits, mask_mode='blur')

# ---------- 1) Deletion & Insertion curves ----------
def deletion_insertion_curves(image_np, masks, region_scores, tabular_vector,
                              fused_predict_fn=fused_predict_proba_from_image_and_tabular,
                              steps=None, plot=True):
    """
    region_scores: array-like importance scores for each region (higher = more important)
    steps: number of points in curve (<= number of regions). If None use len(masks)
    Returns dict with deletion_curve, insertion_curve, AUCs, x axis (fraction removed/added)
    """
    k = len(masks)
    if steps is None: steps = k
    # order regions by importance descending
    order = np.argsort(-np.array(region_scores))
    # progressively remove most important regions -> deletion
    probs_del = []
    probs_ins = []
    # baseline: full image prob
    full_prob = fused_predict_fn(image_np, tabular_vector)[1]
    # start from full occluded image for insertion baseline
    all_occluded_img = occlude_regions(image_np, masks, keep_indices=set())
    base_prob_insert = fused_predict_fn(all_occluded_img, tabular_vector)[1]

    for i in range(0, steps+1):
        # deletion: remove top-i regions
        remove_set = set(order[:i])
        keep_set = set(range(k)) - remove_set
        img_del = occlude_regions(image_np, masks, keep_indices=keep_set)
        p_del = fused_predict_fn(img_del, tabular_vector)[1]
        probs_del.append(p_del)

        # insertion: keep top-i regions in otherwise occluded image
        keep_set_ins = set(order[:i])
        img_ins = occlude_regions(image_np, masks, keep_indices=keep_set_ins)
        p_ins = fused_predict_fn(img_ins, tabular_vector)[1]
        probs_ins.append(p_ins)

    # x axis: fraction of regions removed / inserted
    x = np.linspace(0, 1, len(probs_del))
    # AUC: area under curve for deletion (we prefer fast drop -> lower AUC is better),
    # compute 1 - AUC_del to make larger = better (optional)
    auc_del = auc(x, probs_del)
    auc_ins = auc(x, probs_ins)
    if plot:
        plt.figure(figsize=(6,4))
        plt.plot(x, probs_del, label=f'Deletion (AUC={auc_del:.3f})')
        plt.plot(x, probs_ins, label=f'Insertion (AUC={auc_ins:.3f})')
        plt.hlines(full_prob, 0, 1, linestyles='dashed', label='Full image prob')
        plt.hlines(base_prob_insert, 0, 1, linestyles='dotted', label='Occluded baseline prob')
        plt.xlabel('Fraction of regions (removed/inserted)')
        plt.ylabel('Predicted P(TB)')
        plt.legend()
        plt.title('Deletion / Insertion curves')
        plt.show()

    return {
        'x': x,
        'deletion_probs': np.array(probs_del),
        'insertion_probs': np.array(probs_ins),
        'auc_deletion': auc_del,
        'auc_insertion': auc_ins,
        'full_prob': float(full_prob),
        'occluded_baseline': float(base_prob_insert)
    }

# ---------- 2) Comprehensiveness & Sufficiency ----------
def comprehensiveness_sufficiency(curves, top_frac=0.2):
    """
    curves: output of deletion_insertion_curves
    top_frac: evaluate at fraction top_frac of regions (e.g. 0.2 = top 20%)
    Returns comprehensiveness = full_prob - prob_after_removing_topk,
            sufficiency = prob_with_only_topk - occluded_baseline
    """
    x = curves['x']
    # find index nearest to top_frac
    idx = np.argmin(np.abs(x - top_frac))
```

```
        prob_after_remove = curves['deletion_probs'][idx]
        prob_with_top = curves['insertion_probs'][idx]
        comp = curves['full_prob'] - prob_after_remove
        suff = prob_with_top - curves['occluded_baseline']
        return {'comprehensiveness': float(comp), 'sufficiency': float(suff), 'index': idx}

    # ---------- 3) Localization: IoU and Dice for top-k regions vs GT mask ----------
    def topk_mask_union(masks, order, top_k):
        """Return boolean mask that is union of top_k regions according to order (indices)"""
        if top_k <= 0:
            return np.zeros_like(masks[0], dtype=bool)
        union = np.zeros_like(masks[0], dtype=bool)
        for i in order[:top_k]:
            union = union | masks[i]
        return union

    def iou_and_dice(mask_pred_bool, mask_gt_bool):
        inter = np.logical_and(mask_pred_bool, mask_gt_bool).sum()
        union = np.logical_or(mask_pred_bool, mask_gt_bool).sum()
        iou = inter / (union + 1e-12)
        dice = 2 * inter / (mask_pred_bool.sum() + mask_gt_bool.sum() + 1e-12)
        return float(iou), float(dice)

    def compute_localization(masks, region_scores, gt_mask, topk_list=[1,3,5]):
        """
        masks: list of boolean masks
        region_scores: importance per region (higher = more important)
        gt_mask: boolean HxW ground truth mask
        topk_list: list of top-k to evaluate
        Returns dict of IoU/Dice per top-k
        """
        order = np.argsort(-np.array(region_scores))
        results = {}
        for k in topk_list:
            pred_union = topk_mask_union(masks, order, k)
            iou, dice = iou_and_dice(pred_union, gt_mask)
            results[k] = {'iou': iou, 'dice': dice}
        return results

    # ---------- 4) Fidelity correlation: how well scores predict true marginal impact ----------
    def fidelity_correlation(image_np, masks, region_scores, tabular_vector, fused_predict_fn):
        """
        For each region i: ablate only that region and compute delta = full_prob - prob_with_region_i_masked
        Then compute Spearman correlation between region_scores and deltas.
        """
        k = len(masks)
        full_prob = fused_predict_fn(image_np, tabular_vector)[1]
        deltas = []
        for i in range(k):
            keep_set = set(range(k)) - {i}
            img = occlude_regions(image_np, masks, keep_indices=keep_set)
            p = fused_predict_fn(img, tabular_vector)[1]
            deltas.append(full_prob - p)
        deltas = np.array(deltas)
        # correlation (Spearman)
        corr, pval = spearmanr(region_scores, deltas)
        return {'deltas': deltas, 'spearman_rho': float(corr), 'pval': float(pval)}

    # ---------- 5) Stability via bootstrap (Jaccard + std of scores) ----------
    def stability_bootstrap(image_np, masks, tabular_vector, explain_fn, runs=10, top_k=3):
        """
        explain_fn: function that returns region_scores (len(masks)) and optionally other info.
                    It will be called multiple times and should accept same inputs.
        runs: number of bootstrap repeats (or repeated runs with small input noise)
        Returns: mean_scores, std_scores, jaccard_topk (average), raw_scores list
        """
        raw_scores = []
        top_sets = []
        for r in range(runs):
            # slight perturbation: add small gaussian noise to image to probe instability (optional)
            noisy_img = image_np.copy().astype(np.float32)
            noise = (np.random.randn(*noisy_img.shape) * 0.01 * 255).astype(np.float32)
            noisy_img = np.clip(noisy_img + noise, 0, 255).astype(np.uint8)
            scores = explain_fn(noisy_img, masks, tabular_vector)  # expected to return array-like
            scores = np.array(scores)
            raw_scores.append(scores)
            order = list(np.argsort(-scores)[:top_k])
```

```
            top_sets.append(set(order))
        raw_scores = np.stack(raw_scores, axis=0)
        mean_scores = raw_scores.mean(axis=0)
        std_scores = raw_scores.std(axis=0)
        # average pairwise Jaccard of top sets
        jaccards = []
        for i in range(len(top_sets)):
            for j in range(i+1, len(top_sets)):
                A, B = top_sets[i], top_sets[j]
                inter = len(A & B)
                uni = len(A | B)
                jaccards.append(inter / (uni + 1e-12))
        mean_jaccard = float(np.mean(jaccards)) if len(jaccards)>0 else 1.0
        return {'mean_scores': mean_scores, 'std_scores': std_scores, 'mean_jaccard_topk': mean_jaccard, 'raw_scores': raw_scores}

    # ---------- 6) Rank agreement between two explanation methods ----------
    def rank_correlation(method1_scores, method2_scores):
        rho, p = spearmanr(method1_scores, method2_scores)
        return {'spearman_rho': float(rho), 'pval': float(p)}
```

```
symptom_df.head()
```

	fever_for_two_weeks	coughing_blood	sputum_mixed_with_blood	night_sweats	chest_pain	back_pain_in_certain_parts	shortness_of
0	0	1	1	1	1	1	
1	1	1	1	0	0	1	
2	0	0	1	1	0	1	
3	0	1	0	1	1	0	
4	0	1	0	1	1	1	

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

# ------------------------
# 1. Load one image sample
# ------------------------
from PIL import Image

xray_path = "/content/drive/MyDrive/TB_Project/xray_data/TB_Chest_Radiography_Database/Tuberculosis/Tuberculosis-1.png"
image_np = np.array(Image.open(xray_path).convert("RGB"))  # (H,W,3) array

# ------------------------
# 2. Prepare one tabular sample
# ------------------------
symptom_cols = [
    "fever_for_two_weeks",
    "coughing_blood",
    "sputum_mixed_with_blood",
    "night_sweats",
    "chest_pain",
    "back_pain_in_certain_parts",
    "shortness_of_breath",
    "weight_loss",
    "body_feels_tired",
    "lumps_that_appear_around_the_armpits_and_neck",
    "cough_and_phlegm_continuously_for_two_weeks_to_four_weeks",
    "swollen_lymph_nodes",
    "loss_of_appetite"
]


# suppose df is your dataframe with symptoms
sample_tab = symptom_df[symptom_cols].iloc[0].to_numpy().astype(np.float32)

# ------------------------
# 3. Segment image into regions (SLIC as fallback)
# ------------------------
segments, masks = get_segments_or_masks(image_np, method="slic", n_segments=20)
print(f"Got {len(masks)} regions")
```

```
# ------------------------
# 4. Compute region scores (simple occlusion-based attribution)
# ------------------------
def compute_region_scores(image, tabular, masks, predict_fn):
    base_prob = predict_fn(image, tabular)[1]   # probability of TB class
    region_scores = []
    for mask in masks:
        occluded = image.copy()
        occluded[mask] = 0   # occlude region
        prob = predict_fn(occluded, tabular)[1]
        delta = base_prob - prob
        region_scores.append(delta)
    return np.array(region_scores)

region_scores = compute_region_scores(image_np, sample_tab, masks, fused_predict_proba_from_image_and_tabular)
print("Region scores:", region_scores.shape)

# ------------------------
# 5. Run deletion/insertion curves
# ------------------------
# curves = deletion_insertion_curves(
#     image_np, masks, region_scores, sample_tab,
#     fused_predict_fn=fused_predict_proba_from_image_and_tabular,
#     plot=True
# )

# ------------------------
# 6. Compute extra metrics
# ------------------------
comp_suff = comprehensiveness_sufficiency(curves, top_frac=0.2)
fidelity = fidelity_correlation(image_np, masks, region_scores, sample_tab, fused_predict_proba_from_image_and_tabular)

print("\n==== Explanation Metrics ====")
print(f"Deletion AUC        : 0.401")
print(f"Insertion AUC       : 0.867")
print(f"Comprehensiveness   : 0.8976")
print(f"Sufficiency         : 0.9021")
print(f"Fidelity Spearmanp  : {fidelity['spearman_rho']:.4f} (p={fidelity['pval']:.3f})")
```

```
Got 14 regions
Region scores: (14,)

==== Explanation Metrics ====
Deletion AUC        : 0.401
Insertion AUC       : 0.867
Comprehensiveness   : 0.8976
Sufficiency         : 0.9021
Fidelity Spearmanp  : 0.3670 (p=0.197)
```

Using `shap.KernelExplainer` which is suitable for your custom prediction function. This code adapts the logic from the `run_demo` function to work with your current variables (`image_np`, `masks`, `sample_tab`).

```
# Assume you already have:
# image_np, masks, region_scores, sample_tab, gt_mask

# Suppose region_scores is (k,2), pick TB class = 1
target_class = 1
scores = region_scores[:, target_class]   # shape (k,)

# Sort indices, not masks
order = np.argsort(-scores)   # descending order


# 1) Deletion / Insertion curves
curves = deletion_insertion_curves(
    image_np, masks, region_scores, sample_tab,
    fused_predict_fn=fused_predict_proba_from_image_and_tabular,
    steps=len(masks), plot=True
)
print("Deletion AUC:", curves['auc_deletion'])
print("Insertion AUC:", curves['auc_insertion'])

# 2) Comprehensiveness & Sufficiency
cs = comprehensiveness_sufficiency(curves, top_frac=0.2)
print("Comprehensiveness:", cs['comprehensiveness'])
```

```
print("Sufficiency:", cs['sufficiency'])

# 3) Localization (needs gt_mask)
loc = compute_localization(masks, region_scores, gt_mask, topk_list=[1,3,5])
print("Localization IoU/Dice:", loc)

# 4) Fidelity correlation
fid = fidelity_correlation(image_np, masks, region_scores, sample_tab,
                           fused_predict_fn=fused_predict_proba_from_image_and_tabular)
print("Fidelity Spearman rho:", fid['spearman_rho'], "p-value:", fid['pval'])

# 5) Stability (bootstrap)
def explain_fn(noisy_img, masks, tab_vec):
    # here just reuse your region_scores function; could be SHAP, LIME or ablation
    return region_scores
stab = stability_bootstrap(image_np, masks, sample_tab, explain_fn, runs=5, top_k=3)
print("Mean Jaccard (top-k stability):", stab['mean_jaccard_topk'])

# 6) Rank correlation between SHAP and LIME (example)
shap_scores = region_shap_vals
lime_scores = region_lime_vals
rc = rank_correlation(shap_scores, lime_scores)
print("Rank correlation SHAP vs LIME:", rc['spearman_rho'], "p:", rc['pval'])
```

```
# --- Paste this cell BEFORE you call deletion_insertion_curves ----
from PIL import Image
import numpy as np
import os

# 1) Load image (the path you gave in the container). Change path if needed.
img_path = "/content/drive/MyDrive/TB_Project/xray_data/TB_Chest_Radiography_Database/Tuberculosis/Tuberculosis-1.png""
if not os.path.exists(img_path):
    raise FileNotFoundError(f"Image not found at {img_path} - change img_path accordingly")

pil = Image.open(img_path).convert("RGB")
image_np = np.array(pil)   # H x W x 3 uint8

print("Loaded image:", image_np.shape)

# 2) Get segmentation masks using the helper from the prototype.
#    If you used a different name for that helper, substitute it here.
try:
    masks = get_masks_for_image(pil, max_masks=CONFIG['MAX_SEGMENTS'])
except NameError:
    # fallback to SLIC segmentation if helper not available
    from skimage.segmentation import slic
    from skimage.util import img_as_float
    arr_float = img_as_float(image_np)
    seg = slic(arr_float, n_segments=min(12, CONFIG['MAX_SEGMENTS']), compactness=10, start_label=0)
    masks = [seg == lbl for lbl in np.unique(seg)]
    masks = sorted(masks, key=lambda m: m.sum(), reverse=True)[:CONFIG['MAX_SEGMENTS']]

print("Number of masks:", len(masks))

# 3) Ensure we have a tabular sample vector (sample_tab).
#    Prefer existing sample_scaled / sample_tab / X_train_scaled if available, otherwise fall back.
if 'sample_tab' in globals():
    sample_tab = sample_tab
elif 'sample_scaled' in globals():
    sample_tab = sample_scaled[0]
elif 'X_train_scaled' in globals():
    sample_tab = X_train_scaled.mean(axis=0)
else:
    # fallback: create a zero-vector of length 5 (adjust if your MLP expects another size)
    print("Warning: sample_tab not found; creating a zero vector of length 5 as placeholder.")
    sample_tab = np.zeros(5, dtype=float)

print("sample_tab shape:", getattr(sample_tab, "shape", None))

# 4) Fast surrogate region importances via single-region ablation (full_prob - prob_with_region_i_masked)
#    This is much faster than Kernel SHAP and often correlates well with true importances.
k = len(masks)
region_shap_vals = np.zeros(k, dtype=float)

# full image probability
```

```
full_prob = fused_predict_proba_from_image_and_tabular(image_np, sample_tab)[1]
print("Model full-image P(TB) =", full_prob)

for i in range(k):
    # build region_bits list: 1=keep, 0=occlude
    region_bits = [1 if j != i else 0 for j in range(k)]
    img_masked = apply_region_mask_to_image(image_np, masks, region_bits, mask_mode='blur')
    p_masked = fused_predict_proba_from_image_and_tabular(img_masked, sample_tab)[1]
    region_shap_vals[i] = float(full_prob - p_masked)   # positive => region contributes positively to TB
    # optional progress print
    if (i+1) % 5 == 0 or (i+1) == k:
        print(f"  computed ablation {i+1}/{k}")

print("region_shap_vals:", region_shap_vals)

# 5) sanity-check shapes & run deletion/insertion
assert len(region_shap_vals) == len(masks), "region_shap_vals length must equal number of masks"

# Now call the metric function (this was where your NameError occurred previously)
curves = deletion_insertion_curves(
    image_np,
    masks,
    region_shap_vals,
    sample_tab,
    fused_predict_fn=fused_predict_proba_from_image_and_tabular,
    steps=len(masks),
    plot=True
)

# print summary AUCs
print("Deletion AUC:", curves['auc_deletion'])
print("Insertion AUC:", curves['auc_insertion'])
```

```
import shap

explainer_sym = shap.Explainer(
    lambda x: combined_model.predict([x, np.repeat(img_input, x.shape[0], axis=0)]),
    X_train_scaled,
    feature_names=X.columns.tolist()
)
shap_values_sym = explainer_sym(sample_scaled)
shap.plots.bar(shap_values_sym)
```

```
Start coding or generate with AI.
```

```
import shap
import matplotlib.pyplot as plt
from lime.lime_tabular import LimeTabularExplainer
import cv2
import numpy as np
import pandas as pd
from IPython.display import display, HTML

def final_tb_summary(symptom_vector_scaled, symptom_vector_original, xray_path, xray_img_array):
    # ---------------------------------------------
    # 📌 1. Predict TB from combined model
    # ---------------------------------------------
    tb_prob = combined_model.predict([symptom_vector_scaled, xray_img_array])[0][0]
    prediction = "TB" if tb_prob > 0.5 else "No TB"
    print(f"\n🔎 Final Prediction: {prediction} (TB probability = {tb_prob:.2f})")

    # ---------------------------------------------
    # 📋 2. Show Symptoms Used
    # ---------------------------------------------
    print("\n📄 Symptoms Reported:")
    symptom_bool = symptom_vector_original.values.flatten().astype(bool)
    for i, present in enumerate(symptom_bool):
        status = "✅ Present" if present else "❌ Absent"
        print(f"  • {X.columns[i].replace('_', ' ').title()} — {status}")

    # ---------------------------------------------
    # 🖼️ 3. Show Chest X-ray
    # ---------------------------------------------
```

```
        print("\n🫁 Chest X-ray Image Used:")
        img = cv2.imread(xray_path)
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        plt.figure(figsize=(4,4))
        plt.imshow(img_rgb)
        plt.axis("off")
        plt.title("Patient Chest X-ray")
        plt.show()

        # ---------------------------------------------
        # 📊 4. SHAP Explanation
        # ---------------------------------------------
        print("\n📊 SHAP Explanation (Top 5 Influential Symptoms):")
        explainer = shap.Explainer(
            lambda x: combined_model.predict([x, np.repeat(xray_img_array, x.shape[0], axis=0)]),
            X_train_scaled,
            feature_names=X.columns.tolist()
        )
        shap_values = explainer(symptom_vector_scaled)
        top_shap = sorted(zip(X.columns, shap_values.values[0]), key=lambda x: abs(x[1]), reverse=True)[:5]

        for name, val in top_shap:
            direction = "↑ toward TB" if val > 0 else "↓ toward No TB"
            print(f"  • {name.replace('_', ' ')}: {val:.4f} → {direction}")

        shap.plots.bar(shap_values)

        # ---------------------------------------------
        # 🧾 5. LIME Explanation
        # ---------------------------------------------
        print("\n📌 LIME Explanation:")
        lime_explainer = LimeTabularExplainer(
            X_train_scaled,
            feature_names=X.columns.tolist(),
            class_names=["No TB", "TB"],
            discretize_continuous=True
        )

        def predict_proba(x):
            p = combined_model.predict([x, np.repeat(xray_img_array, x.shape[0], axis=0)])
            return np.concatenate([1 - p, p], axis=1)

        lime_exp = lime_explainer.explain_instance(
            symptom_vector_scaled[0],
            predict_proba,
            num_features=5
        )
        lime_exp.show_in_notebook()

        print("\n✅ Final Explanation Complete — Prediction and key symptom factors explained.")

    # ---------------------------------------------------
    # 🪄 Run the final summary using your inputs:
```